

# Bias-Variance Tradeoff and Regularization

Sourav Karmakar

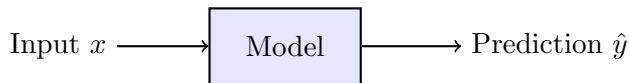
# Outline

1 Bias-Variance Tradeoff

2 Regularization

# Introduction: The Goal of a Model

In machine learning, we train a model on a **training dataset** to predict an output  $\hat{y}$  for a given input  $x$ .



Our ultimate goal is **generalization**: the model should not only perform well on the training data but also on unseen **test data**. Two main sources of error prevent our models from generalizing perfectly: **Bias** and **Variance**.

# What is Bias?

**Bias** is the error introduced by approximating a complex real-world problem by a simpler model. Bias represents the error stemming from overly simplistic assumptions in a learning algorithm. It measures how far off our predicted values are from the true values, *on average*.

## Mathematical Formulation:

$$\text{Bias}(\text{Model}) = \mathbb{E}[\hat{y}] - y$$

Where:

- $\hat{y}$  is the model's prediction.
- $\mathbb{E}[\hat{y}]$  is the *expected* (or average) prediction of the model if we trained it over and over again on many different samples of training data.
- $y$  is the true, actual output.

# Numerical Example: Bias

Let's calculate the bias of a model predicting house prices.

**Scenario:** The true price of a specific house is **\$500k** ( $y = 500$ ). We train our model on 3 different random samples of data and predict the price for this house.

# Numerical Example: Bias

Let's calculate the bias of a model predicting house prices.

**Scenario:** The true price of a specific house is **\$500k** ( $y = 500$ ). We train our model on 3 different random samples of data and predict the price for this house.

## Step 1: Get predictions ( $\hat{y}$ )

- Dataset 1 Prediction: 300
- Dataset 2 Prediction: 320
- Dataset 3 Prediction: 280

# Numerical Example: Bias

Let's calculate the bias of a model predicting house prices.

**Scenario:** The true price of a specific house is **\$500k** ( $y = 500$ ). We train our model on 3 different random samples of data and predict the price for this house.

## Step 1: Get predictions ( $\hat{y}$ )

- Dataset 1 Prediction: 300
- Dataset 2 Prediction: 320
- Dataset 3 Prediction: 280

## Step 2: Expected prediction $\mathbb{E}[\hat{y}]$

$$\mathbb{E}[\hat{y}] = \frac{300 + 320 + 280}{3} = \mathbf{300}$$

# Numerical Example: Bias

Let's calculate the bias of a model predicting house prices.

**Scenario:** The true price of a specific house is **\$500k** ( $y = 500$ ). We train our model on 3 different random samples of data and predict the price for this house.

## Step 1: Get predictions ( $\hat{y}$ )

- Dataset 1 Prediction: 300
- Dataset 2 Prediction: 320
- Dataset 3 Prediction: 280

## Step 2: Expected prediction $\mathbb{E}[\hat{y}]$

$$\mathbb{E}[\hat{y}] = \frac{300 + 320 + 280}{3} = \mathbf{300}$$

## Step 3: Calculate Bias

$$\text{Bias} = 300 - 500 = \mathbf{-200}$$



# Numerical Example: Bias

Let's calculate the bias of a model predicting house prices.

**Scenario:** The true price of a specific house is **\$500k** ( $y = 500$ ). We train our model on 3 different random samples of data and predict the price for this house.

## Step 1: Get predictions ( $\hat{y}$ )

- Dataset 1 Prediction: 300
- Dataset 2 Prediction: 320
- Dataset 3 Prediction: 280

## Step 2: Expected prediction $\mathbb{E}[\hat{y}]$

$$\mathbb{E}[\hat{y}] = \frac{300 + 320 + 280}{3} = \mathbf{300}$$

## Step 3: Calculate Bias

$$\text{Bias} = 300 - 500 = \mathbf{-200}$$

*Conclusion:* This model has a **High Bias**. It consistently underestimates the true value by a large margin.

# What is Variance?

**Variance** is the amount that the model's prediction changes if we train it on a different training dataset. It measures the *inconsistency* or variability of the predictions.

## Mathematical Formulation:

$$\text{Variance} = \mathbb{E} [(\hat{y} - \mathbb{E}[\hat{y}])^2]$$

We want to reduce variance because we want our model to be **robust**—it should not drastically change its predictions just because the training data changed slightly.

# Numerical Example: Variance

Let's calculate the variance for two different models predicting the same \$500k house.

**Model A (From previous slide):**

Predictions: 300, 320, 280

Mean ( $\mathbb{E}[\hat{y}]$ ) = 300

# Numerical Example: Variance

Let's calculate the variance for two different models predicting the same \$500k house.

**Model A (From previous slide):**

Predictions: 300, 320, 280

Mean ( $\mathbb{E}[\hat{y}]$ ) = 300

$$\text{Var}_A = \frac{(300-300)^2 + (320-300)^2 + (280-300)^2}{3}$$

# Numerical Example: Variance

Let's calculate the variance for two different models predicting the same \$500k house.

**Model A (From previous slide):**

Predictions: 300, 320, 280

Mean ( $\mathbb{E}[\hat{y}]$ ) = 300

$$\text{Var}_A = \frac{(300-300)^2 + (320-300)^2 + (280-300)^2}{3}$$

$$\text{Var}_A = \frac{0+400+400}{3}$$

# Numerical Example: Variance

Let's calculate the variance for two different models predicting the same \$500k house.

**Model A (From previous slide):**

Predictions: 300, 320, 280

Mean ( $\mathbb{E}[\hat{y}]$ ) = 300

$$\text{Var}_A = \frac{(300-300)^2 + (320-300)^2 + (280-300)^2}{3}$$

$$\text{Var}_A = \frac{0+400+400}{3}$$

$$\text{Var}_A = \mathbf{266.67} \quad (\text{Low Variance})$$

# Numerical Example: Variance

Let's calculate the variance for two different models predicting the same \$500k house.

**Model A (From previous slide):**

Predictions: 300, 320, 280

Mean ( $\mathbb{E}[\hat{y}]$ ) = 300

$$\text{Var}_A = \frac{(300-300)^2 + (320-300)^2 + (280-300)^2}{3}$$

$$\text{Var}_A = \frac{0+400+400}{3}$$

$$\text{Var}_A = \mathbf{266.67} \quad (\text{Low Variance})$$

**Model B (A new, complex model):**

Predictions: 100, 900, 500

Mean = 500 (*Low Bias*:  $500 - 500 = 0$ )

# Numerical Example: Variance

Let's calculate the variance for two different models predicting the same \$500k house.

**Model A (From previous slide):**

Predictions: 300, 320, 280

Mean ( $\mathbb{E}[\hat{y}]$ ) = 300

$$\text{Var}_A = \frac{(300-300)^2 + (320-300)^2 + (280-300)^2}{3}$$

$$\text{Var}_A = \frac{0+400+400}{3}$$

$$\text{Var}_A = \mathbf{266.67} \quad (\text{Low Variance})$$

**Model B (A new, complex model):**

Predictions: 100, 900, 500

Mean = 500 (*Low Bias*:  $500 - 500 = 0$ )

$$\text{Var}_B = \frac{(100-500)^2 + (900-500)^2 + (500-500)^2}{3}$$



# Numerical Example: Variance

Let's calculate the variance for two different models predicting the same \$500k house.

**Model A (From previous slide):**

Predictions: 300, 320, 280

Mean ( $\mathbb{E}[\hat{y}]$ ) = 300

$$\text{Var}_A = \frac{(300-300)^2 + (320-300)^2 + (280-300)^2}{3}$$

$$\text{Var}_A = \frac{0+400+400}{3}$$

$$\text{Var}_A = \mathbf{266.67} \quad (\text{Low Variance})$$

**Model B (A new, complex model):**

Predictions: 100, 900, 500

Mean = 500 (*Low Bias*:  $500 - 500 = 0$ )

$$\text{Var}_B = \frac{(100-500)^2 + (900-500)^2 + (500-500)^2}{3}$$

$$\text{Var}_B = \frac{(-400)^2 + (400)^2 + 0}{3}$$

# Numerical Example: Variance

Let's calculate the variance for two different models predicting the same \$500k house.

**Model A (From previous slide):**

Predictions: 300, 320, 280

Mean ( $\mathbb{E}[\hat{y}]$ ) = 300

$$\text{Var}_A = \frac{(300-300)^2 + (320-300)^2 + (280-300)^2}{3}$$

$$\text{Var}_A = \frac{0+400+400}{3}$$

$$\text{Var}_A = \mathbf{266.67} \quad (\text{Low Variance})$$

**Model B (A new, complex model):**

Predictions: 100, 900, 500

Mean = 500 (*Low Bias*:  $500 - 500 = 0$ )

$$\text{Var}_B = \frac{(100-500)^2 + (900-500)^2 + (500-500)^2}{3}$$

$$\text{Var}_B = \frac{(-400)^2 + (400)^2 + 0}{3}$$

$$\text{Var}_B \approx \mathbf{106,666} \quad (\text{High Variance})$$

# Numerical Example: Variance

Let's calculate the variance for two different models predicting the same \$500k house.

**Model A (From previous slide):**

Predictions: 300, 320, 280

Mean ( $\mathbb{E}[\hat{y}]$ ) = 300

$$\text{Var}_A = \frac{(300-300)^2 + (320-300)^2 + (280-300)^2}{3}$$

$$\text{Var}_A = \frac{0+400+400}{3}$$

$$\text{Var}_A = \mathbf{266.67} \quad (\text{Low Variance})$$

**Model B (A new, complex model):**

Predictions: 100, 900, 500

Mean = 500 (*Low Bias*:  $500 - 500 = 0$ )

$$\text{Var}_B = \frac{(100-500)^2 + (900-500)^2 + (500-500)^2}{3}$$

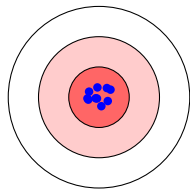
$$\text{Var}_B = \frac{(-400)^2 + (400)^2 + 0}{3}$$

$$\text{Var}_B \approx \mathbf{106,666} \quad (\text{High Variance})$$

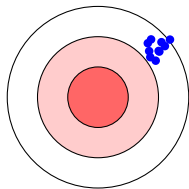
*Conclusion:* Model B has zero bias but **High Variance**. Its predictions are wildly unstable depending on the training data.

# Visualizing Bias and Variance (The Dart Board Diagram)

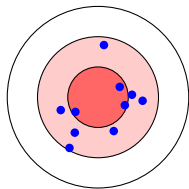
Imagine the center of the target (bullseye) in the dart board is the true value  $y$ . The dots are predictions from models trained on different training samples.



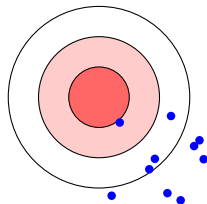
**Low Bias**  
**Low Variance**  
(Ideal)



**High Bias**  
**Low Variance**



**Low Bias**  
**High Variance**



**High Bias**  
**High Variance**

# The Total Error Equation

The expected Mean Squared Error (MSE) of a model on unseen data can be mathematically decomposed into three parts:

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

- **Bias<sup>2</sup>:** Error due to wrong assumptions in the model.
- **Variance:** Error due to the model's sensitivity to small fluctuations in the training set.
- **Irreducible Error ( $\epsilon$ ):** The noise inherent in the problem itself. No model can ever reduce this error.

# High Bias Causes and Effects

## What causes High Bias?

- Using overly **simpler models** (e.g., fitting a straight line to curved data).
- Having too few features.

## Effects on Error:

- **Training Error:** HIGH (The model cannot even capture the underlying pattern in the training data).
- **Test Error:** HIGH.

This situation is famously known as **Underfitting**. The model is too rigid to learn anything useful.

# High Variance Causes and Effects

## What causes High Variance?

- Using very **complex models** (e.g., a high-degree polynomial curve).
- Having too many features compared to the number of data points.

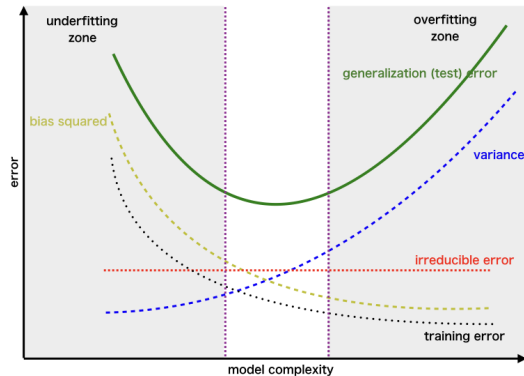
## Effects on Error:

- **Training Error:** LOW (The model perfectly memorizes the training data, including the random noise).
- **Test Error:** HIGH (Because it memorized noise, it fails completely on new data).

This situation is famously known as **Overfitting**.

# The Bias-Variance Tradeoff Curve

We cannot minimize both Bias and Variance simultaneously. Decreasing bias usually increases variance, and vice versa.

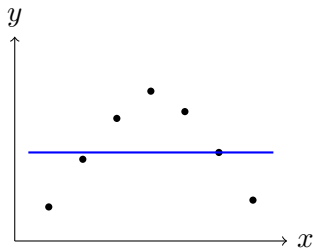


**Goal:** Find the “sweet spot” (optimum complexity) that minimizes the Total Error on the test dataset.

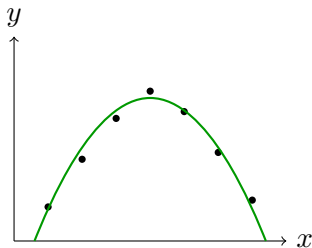


# Underfitting vs. Overfitting Revisited

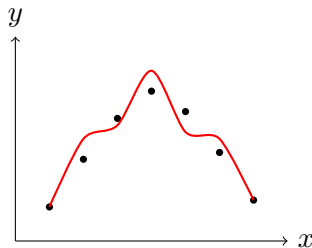
Let's visualize the tradeoff using a dataset.



Underfitting (High Bias)



Optimal Fit



Overfitting (High Variance)

How do we stop an overfitted model from drawing those extreme curves? We use **Regularization**.

# The Role of Model Parameters ( $\theta$ )

In models like **Linear Regression**, the predicted output  $\hat{y}$  is determined by a linear combination of input features and their **weights** (parameters)  $\theta$ :

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_k x_k$$

## Interpretation of $\theta_j$ :

- $\theta_j = \frac{\Delta \hat{y}}{\Delta x_j}$ . It represents the change in response per unit change in feature  $x_j$ .
- If a weight  $\theta_j$  is **very large**, the model's response changes drastically for even tiny fluctuations (noise) in  $x_j$ .
- **Large weights = High Sensitivity = High Variance.**

# What is Regularization?

**Regularization** is a technique to reduce overfitting by limiting (shrinking) the parameters ( $\theta$ ). We force the model to keep the weights small.

We do this by introducing a “**Penalty**” term into the standard Loss function:

$$L(\Theta, \alpha) = \text{Loss}(\Theta) + \alpha \cdot (\text{Penalty})$$

- **Loss( $\Theta$ ):** Can be MSE (Regression), BCE, or CCE (Classification).
- **Penalty:** A function of the model weights ( $f(\theta_1, \theta_2 \dots)$ ).
- **$\alpha$  (Regularization Parameter):** Controls how much penalty is applied.
  - If  $\alpha = 0$ : No penalty (Unregularized).
  - If  $\alpha$  is high: Heavy penalty (Weights approach 0).

# L2 Regularization and Ridge Regression

**L2 Regularization** is a general technique that adds the squared magnitude of the weights (L2-norm) as a penalty to *any* loss function.

**Ridge Regression** is the specific model created when we apply L2 Regularization to standard Linear Regression.

$$L(\Theta, \alpha) = \underbrace{\frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2}_{\text{Standard MSE Loss}} + \underbrace{\frac{\alpha}{2} \sum_{j=1}^k \theta_j^2}_{\text{L2 Penalty Term}}$$

# L2 Regularization and Ridge Regression

**L2 Regularization** is a general technique that adds the squared magnitude of the weights (L2-norm) as a penalty to *any* loss function.

**Ridge Regression** is the specific model created when we apply L2 Regularization to standard Linear Regression.

$$L(\Theta, \alpha) = \underbrace{\frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2}_{\text{Standard MSE Loss}} + \underbrace{\frac{\alpha}{2} \sum_{j=1}^k \theta_j^2}_{\text{L2 Penalty Term}}$$

Why don't we regularize the bias term ( $\theta_0$ )?

# L2 Regularization and Ridge Regression

**L2 Regularization** is a general technique that adds the squared magnitude of the weights (L2-norm) as a penalty to *any* loss function.

**Ridge Regression** is the specific model created when we apply L2 Regularization to standard Linear Regression.

$$L(\Theta, \alpha) = \underbrace{\frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2}_{\text{Standard MSE Loss}} + \underbrace{\frac{\alpha}{2} \sum_{j=1}^k \theta_j^2}_{\text{L2 Penalty Term}}$$

**Why don't we regularize the bias term ( $\theta_0$ )?**

- The bias term  $\theta_0$  simply shifts the function up or down.
- It does not determine the “curvature” or the model’s sensitivity to input features  $x$ .
- Penalizing  $\theta_0$  doesn’t reduce variance; it just forces the curve away from the data’s true baseline.

# Gradient Descent for Ridge Regression

To train the model, we use Gradient Descent. We need the derivative of the new loss function with respect to  $\theta_j$ .

**Step 1: Calculate the Gradient**

# Gradient Descent for Ridge Regression

To train the model, we use Gradient Descent. We need the derivative of the new loss function with respect to  $\theta_j$ .

**Step 1: Calculate the Gradient**

$$\frac{\partial L}{\partial \theta_j} = \left( \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)} \right) + \alpha \theta_j$$



# Gradient Descent for Ridge Regression

To train the model, we use Gradient Descent. We need the derivative of the new loss function with respect to  $\theta_j$ .

**Step 1: Calculate the Gradient**

$$\frac{\partial L}{\partial \theta_j} = \left( \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)} \right) + \alpha \theta_j$$

**Step 2: Update Rule**  $(\theta_j^{(t+1)} \leftarrow \theta_j^{(t)} - \eta \frac{\partial L}{\partial \theta_j})$

# Gradient Descent for Ridge Regression

To train the model, we use Gradient Descent. We need the derivative of the new loss function with respect to  $\theta_j$ .

**Step 1: Calculate the Gradient**

$$\frac{\partial L}{\partial \theta_j} = \left( \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)} \right) + \alpha \theta_j$$

**Step 2: Update Rule**  $(\theta_j^{(t+1)} \leftarrow \theta_j^{(t)} - \eta \frac{\partial L}{\partial \theta_j})$

$$\theta_j^{(t+1)} = \theta_j^{(t)} - \eta \left[ \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)} + \alpha \theta_j^{(t)} \right]$$

# Gradient Descent for Ridge Regression

To train the model, we use Gradient Descent. We need the derivative of the new loss function with respect to  $\theta_j$ .

**Step 1: Calculate the Gradient**

$$\frac{\partial L}{\partial \theta_j} = \left( \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)} \right) + \alpha \theta_j$$

**Step 2: Update Rule**  $(\theta_j^{(t+1)} \leftarrow \theta_j^{(t)} - \eta \frac{\partial L}{\partial \theta_j})$

$$\theta_j^{(t+1)} = \theta_j^{(t)} - \eta \left[ \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)} + \alpha \theta_j^{(t)} \right]$$

**Step 3: Rearrange to see the magic!**

# Gradient Descent for Ridge Regression

To train the model, we use Gradient Descent. We need the derivative of the new loss function with respect to  $\theta_j$ .

**Step 1: Calculate the Gradient**

$$\frac{\partial L}{\partial \theta_j} = \left( \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)} \right) + \alpha \theta_j$$

**Step 2: Update Rule**  $(\theta_j^{(t+1)} \leftarrow \theta_j^{(t)} - \eta \frac{\partial L}{\partial \theta_j})$

$$\theta_j^{(t+1)} = \theta_j^{(t)} - \eta \left[ \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)} + \alpha \theta_j^{(t)} \right]$$

**Step 3: Rearrange to see the magic!**

$$\theta_j^{(t+1)} = (1 - \eta\alpha) \theta_j^{(t)} - \eta [\text{Standard Gradient}]$$

# Gradient Descent for Ridge Regression

To train the model, we use Gradient Descent. We need the derivative of the new loss function with respect to  $\theta_j$ .

**Step 1: Calculate the Gradient**

$$\frac{\partial L}{\partial \theta_j} = \left( \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)} \right) + \alpha \theta_j$$

**Step 2: Update Rule**  $(\theta_j^{(t+1)} \leftarrow \theta_j^{(t)} - \eta \frac{\partial L}{\partial \theta_j})$

$$\theta_j^{(t+1)} = \theta_j^{(t)} - \eta \left[ \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)} + \alpha \theta_j^{(t)} \right]$$

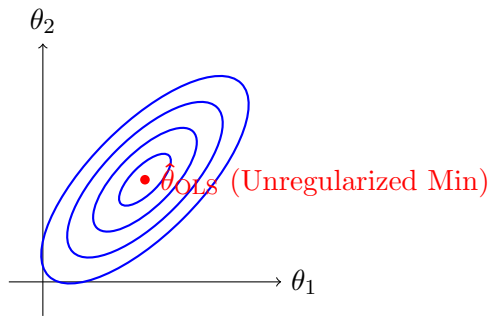
**Step 3: Rearrange to see the magic!**

$$\theta_j^{(t+1)} = (1 - \eta\alpha) \theta_j^{(t)} - \eta [\text{Standard Gradient}]$$

Notice  $(1 - \eta\alpha)$ : Before applying the standard update, we shrink the current weight by a small fraction. This is why L2 regularization is also called **Weight Decay**.

# Visualizing Loss with Contour Diagrams

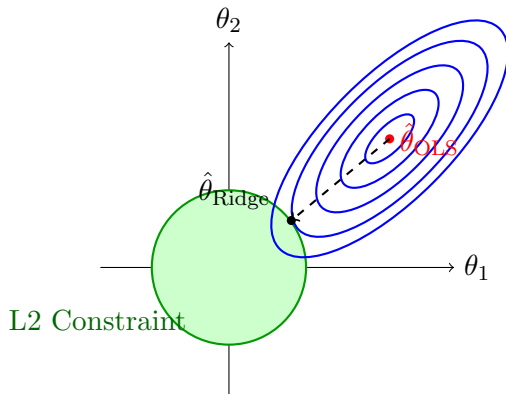
**What is a Contour Diagram?** It is a 2D representation of a 3D bowl-like surface. Each ring (contour) represents points where the Loss value is the same. The center is the minimum loss.



**Unregularized MSE  
Loss Contours**

## L2 Regularization Contours

In Ridge Regression, we want to minimize the MSE, but we are **constrained** by the L2 penalty budget:  $\theta_1^2 + \theta_2^2 \leq t$ . This constraint forms a **Circle**.



*The solution is where the MSE ellipse first touches the L2 circle. Weights are shrunk towards zero, but rarely become exactly zero.*

# L1 Regularization and LASSO

**L1 Regularization** is a general technique that adds the sum of the absolute values of weights (L1-norm) as a penalty to *any* loss function.

**LASSO** (**L**east **A**bsolute **S**hrinkage and **S**election **O**perator) is the specific model created when L1 Regularization is applied to standard Linear Regression.

$$L(\Theta, \alpha) = \underbrace{\frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2}_{\text{Standard MSE Loss}} + \underbrace{\alpha \sum_{j=1}^k |\theta_j|}_{\text{L1 Penalty Term}}$$

## Gradient Descent for LASSO:

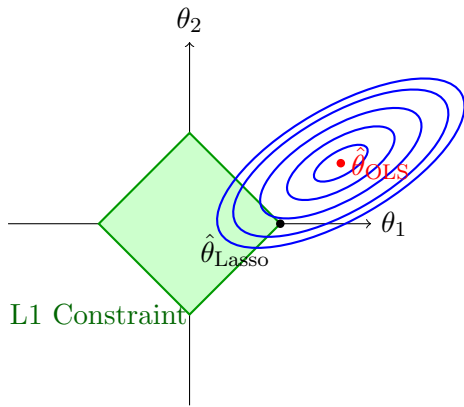
Because  $|\theta_j|$  is not differentiable at 0, the update rule uses the **sign** of  $\theta_j$ :

$$\theta_j^{(t+1)} = \theta_j^{(t)} - \eta \left[ \text{Standard Gradient} + \alpha \cdot \text{sign}(\theta_j^{(t)}) \right]$$



# L1 Regularization Contours and Feature Selection

The L1 constraint ( $|\theta_1| + |\theta_2| \leq t$ ) forms a **Diamond** shape with sharp corners on the axes.



# How LASSO Performs Feature Selection?

## Why does LASSO zero out features?

- Look at the L1 constraint: It has sharp corners that lie **exactly on the axes** (e.g., where  $\theta_2 = 0$ ).
- The expanding elliptical contours of the MSE loss are highly likely to hit these sharp corners first.
- When the intersection happens on an axis, the corresponding weight becomes **exactly zero**.
- By forcing unimportant feature weights to exactly zero, LASSO automatically performs **Feature Selection**, creating sparse, interpretable models.

# The Problem with LASSO: Correlated Features

**Scenario:** Imagine predicting house prices using two highly correlated features, like **Square Footage** ( $x_1$ ) and **Number of Rooms** ( $x_2$ ).

**How LASSO (L1) Handles Them:**

- Because they share predictive power, LASSO views them as redundant.
- The sharp corners of the L1 diamond force the model to arbitrarily assign a large weight to one feature (e.g.,  $\theta_1 > 0$ ) and shrink the other to **exactly zero** ( $\theta_2 = 0$ ).
- **The Flaw:** This makes the model highly **unstable**. A tiny change in the training data might cause the model to completely flip, dropping  $x_1$  and keeping  $x_2$  instead!

# Combined L1 & L2 and Elastic-Net

What if we want the feature selection of L1, but the stability of L2? We can combine both penalties into a general loss function.

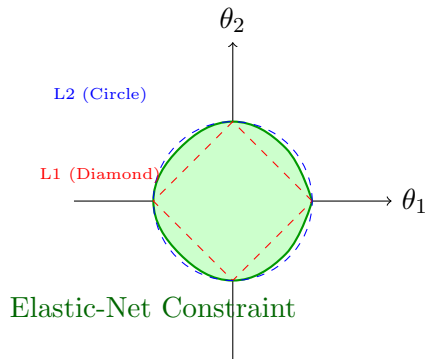
**Elastic-Net** is the specific model created when this combined penalty is applied to standard Linear Regression.

$$L(\Theta, \alpha, \beta) = \underbrace{\frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2}_{\text{Standard MSE Loss}} + \alpha \underbrace{\left[ \beta \sum_{j=1}^k |\theta_j| + \frac{1 - \beta}{2} \sum_{j=1}^k \theta_j^2 \right]}_{\text{Combined Penalty Term}}$$

- $\alpha$ : Overall regularization strength.
- $\beta$ : The **L1 ratio** parameter ( $0 \leq \beta \leq 1$ ).
  - If  $\beta = 0$ : Only L2 is applied (Ridge).
  - If  $\beta = 1$ : Only L1 is applied (Lasso).

# Elastic-Net Contours

The constraint shape of Elastic-Net is a compromise between the sharp diamond of L1 and the smooth circle of L2. It looks like a “bulgy” diamond.



*It retains some sharp corners for feature selection, but the curve provides grouping effects (correlated features are selected or dropped together).*

# The Elastic-Net Benefit: The Grouping Effect

Elastic-Net fixes the instability of LASSO (as explained [in this slide](#)) by combining the L1 diamond with the L2 circle, creating curved edges.

## How Elastic-Net Handles Correlated Features:

- The L2 component heavily penalizes putting 100% of the weight on a single variable.
- Instead of hitting a sharp corner on an axis, the loss contour hits the **curved edge**, distributing the weight across both features ( $\theta_1 \approx \theta_2$ ).
- **The Grouping Effect:** Correlated features are treated as a “package deal.” They are either **selected together** (if useful) or **dropped together** (if useless).
- **The Result:** A sparse model (thanks to L1) that remains highly stable and robust (thanks to L2).

# Do We Always Use Regularization?

**Short Answer: No.** Regularization is a specific cure for **High Variance** (Overfitting).

**When YOU SHOULD use Regularization:**

- **High Dimensionality** ( $k \approx m$  or  $k > m$ ): You have many features compared to the number of data points.
  - *Example:* Predicting a disease using 10,000 genetic markers, but you only have data for 100 patients.
- **Multicollinearity:** Your input features are highly correlated.
  - *Example:* Predicting house prices using both “Square Footage” and “Number of Rooms” (Ridge or Elastic-Net helps stabilize these weights).
- **Clear Overfitting:** Your training error is very low, but your test error is incredibly high.

# When to AVOID Regularization?

## When YOU SHOULD NOT use Regularization:

- **Underfitting (High Bias):** The model is already struggling to learn the underlying pattern. Adding a penalty will just flatten the model further, worsening the bias.
  - *Example:* Trying to fit a simple straight line to highly complex, non-linear data.
- **Massive Datasets with Few Features ( $m \gg k$ ):** If you have millions of rows of data and only 4 features, the sheer volume of data naturally prevents overfitting.



# When to AVOID Regularization?

## When YOU SHOULD NOT use Regularization:

- **Underfitting (High Bias):** The model is already struggling to learn the underlying pattern. Adding a penalty will just flatten the model further, worsening the bias.
  - *Example:* Trying to fit a simple straight line to highly complex, non-linear data.
- **Massive Datasets with Few Features ( $m \gg k$ ):** If you have millions of rows of data and only 4 features, the sheer volume of data naturally prevents overfitting.

***Rule of Thumb:** Always train an unregularized baseline model first. Only apply regularization if you actively observe overfitting!*

# Which regularization to Choose?

- **Ridge (L2) – The Best Default:** Start here! Use when you assume most or all of your features provide at least some useful information.
- **LASSO (L1):** Use when you have a massive amount of features, but you strongly suspect only a handful are actually important.
- **Elastic-Net:** Use when you want feature selection, but your features are highly correlated. It is generally preferred over pure LASSO!

# Summary

- **Bias** (Underfitting): Model is too simple. Train and test errors are both high.
- **Variance** (Overfitting): Model is too complex. Train error is low, but test error is high.
- **Bias Variance Tradeoff**: We must find the optimal complexity to minimize total error on unseen data.
- **Regularization**: Adds a penalty term to the loss function to shrink weights and reduce Variance (overfitting).
- **L2 Regularization & Ridge**: L2 adds a squared penalty (circular contours). Ridge applies this to Linear Regression for smooth weight shrinkage.
- **L1 Regularization & LASSO**: L1 adds an absolute penalty (diamond contours). LASSO applies this to Linear Regression, automatically performing **feature selection**.
- **Combined Penalty & Elastic-Net**: Balances L1 and L2 constraints to achieve both feature selection and model stability.

# Thank You